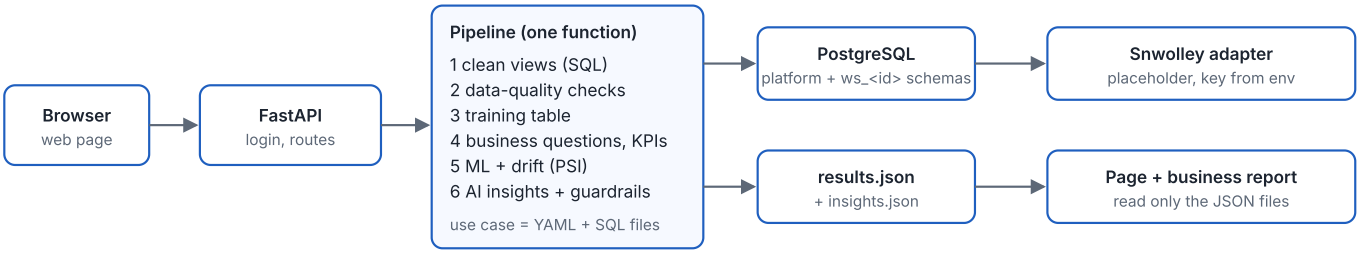
Architecture note

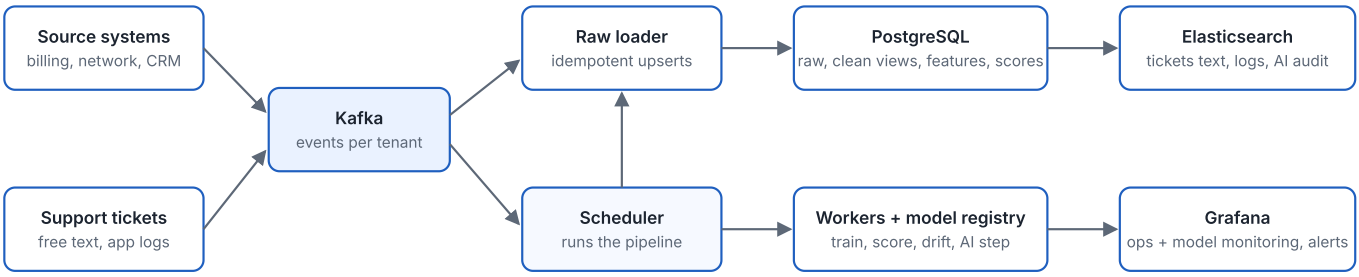
AI-augmented ML analytics platform: what is built, and how it grows into production with Kafka, Elasticsearch, Grafana and scheduled pipelines.

1. What is built (the prototype)



- **Multi-tenancy:** one Postgres schema per workspace (name chosen by the server, never the client); platform tables (users, workspaces, runs) in their own schema. A partial unique index allows only one running analysis per workspace.
- **Generic by configuration:** the platform code never mentions churn or revenue. A use case is a YAML file and SQL files; two ship (classification and regression).
- **Traceability:** `results.json` is the single record of a run. The AI summary and the report read only that record, so every number can be traced to SQL or the ML step.
- **AI behind an adapter:** the Snwolley call lives in one file. The AI only sees a numbered facts sheet; its output is checked (citations, numbers, causal wording, allowed recommendation types) and falls back to a labelled template.

2. Production architecture



Component	Role	Why, and what it is not for
Kafka	Source systems publish events (payment, usage, ticket, plan change), keyed by workspace and customer. A consumer upserts into the raw tables using the event id.	Decouples sources from the platform, buffers bursts, lets us replay history after a bug fix. Event ids make duplicates harmless, which is exactly the duplicate-row problem the clean views handle today. Not used for analytics queries.
Scheduled pipelines	Nightly: load, clean views, **data-quality gate**, features at today's date, score with the registered model, drift check, AI insights, report. Weekly or monthly: retrain as champion vs challenger. Each run is idempotent and recorded in `platform.runs`.	Orchestrator such as Airflow or Prefect, with a job queue (Celery/RQ) replacing the prototype's in-process background task. A failed gate stops scoring and keeps the last good results, marked stale.
PostgreSQL	System of record for raw data, cleaning views and features, plus predictions and run history.	SQL stays central. Schema per workspace works for hundreds of tenants; very large tenants move to their own database. Add a pooler (PgBouncer) and read replicas for reports.
Elasticsearch	Indexes what Postgres is bad at: support-ticket text (search, topic counts, a text feature for models), application and pipeline logs, and an audit trail of every AI summary with its facts and rejections.	Not the analytics store. The current data has no free text, so this is the first thing to add once ticket text is ingested; logs and the AI audit are useful immediately.
Grafana	Dashboards and alerts for the people who run the platform: run success and duration, data-quality counts, PSI per feature over time, score distribution, model metrics once labels arrive, AI accepted / rejected / fallback rates.	Reads Postgres (a monitoring table written by each run) and Elasticsearch (logs). Managers keep reading the business report; Grafana is for operators. Thresholds are in `docs/monitoring_plan.md`.

3. Cross-cutting decisions

- **Security:** secrets in a secrets manager, not files; the AI key is read only by the adapter and never logged. Customer names, emails and phones never reach the AI (it sees aggregated facts only). Per-tenant topics, schemas and index aliases keep tenants apart; the server, not the client, decides the tenant. Auth moves to a shared session store, with throttling in Redis rather than process memory.
- **Reproducibility:** seeds and library versions recorded in the model card; the registry stores model, training data hash and metrics, so any score can be traced to the run that produced it.
- **Failure behaviour:** if the AI provider is down or its answer fails the checks, reports still ship with the template summary, labelled. If data quality fails, scoring stops rather than producing confident nonsense.

- **Cost and scale:** feature SQL is the heavy part (the prototype fixed slow CTEs with `MATERIALIZED` and pre-aggregation); at scale features are computed incrementally per day and stored, and training runs on a worker, not the web process.

4. What I would build next

1) Finish the Snwolley adapter from its documentation. 2) A monitoring table plus a Grafana dashboard fed by each run. 3) Replace the background task with a queue and scheduler. 4) Kafka consumer for one source (payments) to prove the ingestion path. 5) Time-based train/test split and performance monitoring when labels mature.